

STOCKIQ · DOCUMENTAÇÃO DE ML

Machine Learning Mineração de Dados

Documentação do módulo analítico do StockIQ:
previsão de demanda, classificação ABC/XYZ e o
pipeline que conecta o banco operacional à inteligência
do sistema.

Python 3

pandas · NumPy

Séries temporais

CRISP-DM

Previsão de demanda

Gabriel da Silveira Pessoni · Lívia Portela Ferreira

FATEC Franca — Desenvolvimento de Software Multiplataforma ·
6º semestre

Projeto Integrador VI

Computação em Nuvem II · Mineração
de Dados

Sumário

01	Visão geral	Objetivo e papel no sistema
02	Metodologia CRISP-DM	Etapas do ciclo de mineração
03	Base de dados	Dataset e mapeamento para o StockIQ
04	Pipeline de previsão	Do histórico à previsão semanal
05	Algoritmo	Regressão linear sobre séries semanais
06	Implementação	Funções, parâmetros e saída
07	Classificação ABC / XYZ	Valor e regularidade da demanda
08	Técnicas do módulo	Portfólio de técnicas de mineração
09	Pipeline de dados na nuvem	SQS → S3 → modelos → dashboard
10	Integração com o sistema	Backend, banco e tela de IA Analítica
11	Execução	Ambiente, dependências e comandos
12	Evolução do módulo	Roteiro de amadurecimento dos modelos

01 Visão geral

O módulo de Machine Learning transforma o histórico de movimentações do StockIQ em conhecimento acionável: quanto cada categoria deve vender na próxima semana, quais produtos concentram valor e quando repor o estoque.

12SEMANAS DE
HISTÓRICO**1**

SEMANA PREVISTA

3

CLASSES DE TENDÊNCIA

6TÉCNICAS NO
PORTFÓLIO

Este módulo corresponde ao componente curricular de **Mineração de Dados** e se conecta ao restante da solução: os dados nascem nas movimentações do backend, são preparados em Python e retornam como previsões e sugestões na tela de **IA Analítica** do front-end.

**Previsão de demanda**

Estima a quantidade de saída da próxima semana por categoria de produto.

**Curva ABC**

Prioriza produtos pelo valor acumulado de movimentação (já exposta pela API).

**Matriz ABC/XYZ**

Cruza valor com regularidade da demanda para definir políticas de estoque.

**Antecipação de ruptura**

Compara previsão e estoque mínimo/máximo para sugerir reposição.

**Pipeline reproduzível**

Cada etapa é uma função isolada, fácil de testar e de trocar.

**Pronto para a nuvem**

Arquitetura de dados com SQS e S3 definida para escalar a análise.

03 Base de dados

O módulo foi desenvolvido sobre o dataset público **Store Sales — Time Series Forecasting** (Corporación Favorita), da competição do Kaggle. Ele possui vendas diárias por loja e por família de produtos, estrutura praticamente idêntica à das movimentações de saída do StockIQ, o que permite validar o pipeline ponta a ponta e migrar para os dados reais sem mudar o algoritmo.

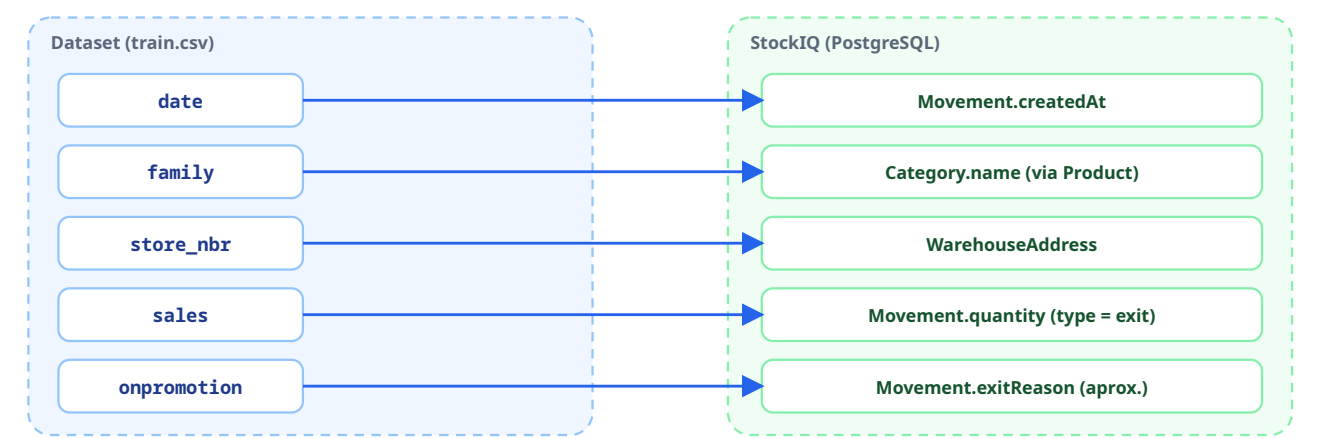


Figura 2 — Mapeamento de colunas do dataset para o modelo de dados do StockIQ

Coluna	Tipo	Conceito no StockIQ	Uso no pipeline
date	data	Movement.createdAt	Eixo temporal; agregação semanal
family	texto	Category.name	Chave de agrupamento (categoria)
sales	numérico	Movement.quantity das saídas	Variável prevista (quantidade de saída)
store_nbr	inteiro	WarehouseAddress	Reservado para análises por local
onpromotion	inteiro	Movement.exitReason	Reservado para análises de causa

ESTRATÉGIA EM DUAS FASES

Fase 1 — prototipação com o dataset público e com a massa simulada do `seed`. **Fase 2** — substituição progressiva pelos dados reais de operação, mantendo a mesma estrutura de atributos. Só a função `load_sales()` muda.

● Atributos de interesse por técnica

Grupo de dados	Atributos-chave	Uso analítico
Movimentações	tipo, quantidade, valor, data, produto, motivo de saída	Séries temporais, curva ABC, detecção de anomalias
Produtos	categoria, marca, estoque mín/máx, preço	Classificação ABC/XYZ e clustering por padrão de consumo
Lotes	datas de fabricação/validade, status	Previsão de perdas por vencimento
Fornecedores	lead time (pedido → entrada)	Otimização do ponto de pedido
Endereços	ocupação, corredor, posição	Associação produto-localização (slotting)

04 Pipeline de previsão

O script `src/forecast_demand.py` organiza o processo em funções pequenas e encadeadas. Cada etapa recebe e devolve um *DataFrame*, o que permite substituir qualquer parte (por exemplo, a fonte de dados) sem tocar nas demais.



Figura 3 — Fluxo do pipeline de previsão

● Por que descartar a última semana?

O corte do dataset costuma cair no meio de uma semana. Somar apenas alguns dias produziria um valor artificialmente baixo e puxaria a tendência para baixo. A etapa `weekly_demand()` remove a semana mais recente, garantindo que todas as janelas usadas no ajuste estejam completas.

05 Algoritmo

A previsão é feita com **regressão linear simples** sobre a série semanal de cada categoria. É um baseline intencionalmente transparente: fácil de explicar, rápido de executar e estável para estabelecer a referência contra a qual modelos mais sofisticados serão comparados.

Passo a passo

1. Para cada categoria, ordenar as semanas e tomar as **últimas 12** (`WEEKS_HISTORY`).
2. Definir $x = 0, 1, \dots, n-1$ (índice da semana) e y = quantidade de saída semanal.
3. Ajustar a reta $y = a \cdot x + b$ por mínimos quadrados (`numpy.polyfit` , grau 1).
4. Projetar a semana seguinte: $\text{previsão} = a \cdot n + b$, com piso em zero.
5. Classificar a tendência pelo coeficiente angular a .

Classificação de tendência

Tendência	Regra (inclinação a)	Leitura de negócio
alta	$a > 0,5$	Demanda crescente — avaliar aumento de estoque mínimo e antecipar compras
estavel	$-0,5 \leq a \leq 0,5$	Demanda regular — manter política atual
queda	$a < -0,5$	Demanda em retração — evitar excesso de estoque

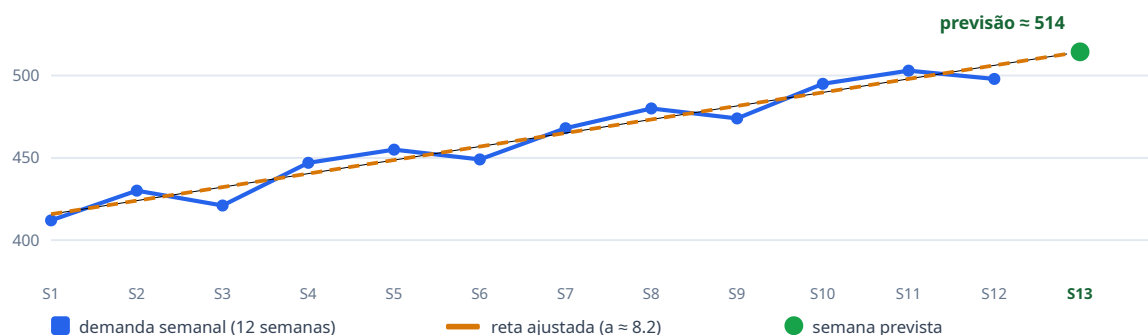


Figura 4 — Ilustração do método com uma série fictícia de uma categoria (valores ilustrativos)

● Por que um baseline linear?



Interpretável

A inclinação diz diretamente se a demanda sobe, cai ou está estável.



Leve

Executa em segundos sobre milhões de linhas por agregação semanal.



Referência

Estabelece o desempenho mínimo que outros modelos precisam superar.

06 Implementação

● Parâmetros e caminhos

Constante	Valor	Função
<code>BASE_DIR</code>	raiz de <code>machine-learning/</code>	Base dos caminhos relativos
<code>RAW_FILE</code>	<code>data/raw/train.csv</code>	Histórico diário de vendas
<code>OUTPUT_FILE</code>	<code>data/previsao_demanda.csv</code>	Arquivo de saída
<code>WEEKS_HISTORY</code>	12	Janela de semanas usada no ajuste

● Funções

Função	Entrada → Saída	Descrição
<code>load_sales()</code>	CSV → DataFrame(date, categoria, quantidade_saida)	Lê somente as colunas necessárias e renomeia para o vocabulário do StockIQ. É o ponto de troca para a consulta SQL na tabela <code>movements</code>
<code>weekly_demand(df)</code>	DataFrame → DataFrame semanal	Agrupar por categoria, faz <i>resample</i> semanal (soma) e remove a semana incompleta
<code>forecast_next_week(weekly)</code>	DataFrame semanal → DataFrame de previsões	Ajusta a reta por categoria, calcula previsão e tendência e ordena por demanda prevista
<code>main()</code>	—	Orquestra o pipeline, imprime o resultado e grava o CSV

● Núcleo do modelo

```
def forecast_next_week(weekly):
    for categoria, grupo in weekly.groupby("categoria"):
        grupo = grupo.sort_values("date").tail(WEEKS_HISTORY)
        if len(grupo) < 2: continue

        x = np.arange(len(grupo))
        y = grupo["quantidade_saida"].to_numpy()
        inclinacao, intercepto = np.polyfit(x, y, 1)
        previsao = max(0.0, inclinacao * len(grupo) + intercepto)

        tendencia = "alta" if inclinacao > 0.5 else "queda" if inclinacao < -0.5 else "estavel"
```

● Saída gerada

Coluna	Descrição
<code>categoria</code>	Categoria de produto
<code>demanda_media_semanal</code>	Média das últimas 12 semanas (1 casa decimal)
<code>previsao_proxima_semana</code>	Quantidade de saída projetada, nunca negativa (1 casa decimal)
<code>tendencia</code>	<code>alta</code> · <code>estavel</code> · <code>queda</code>

O arquivo é ordenado por `previsao_proxima_semana` em ordem decrescente, de modo que as categorias de maior demanda aparecem primeiro. A pasta `data/` fica fora do controle de versão porque os CSVs somam mais de 100 MB e são regeneráveis a partir do dataset original.

07 Classificação ABC / XYZ

Duas visões complementares ajudam a decidir *quanto* e *como* controlar cada item: a **curva ABC** mede importância financeira e a **classificação XYZ** mede a regularidade do consumo.

● Curva ABC — já disponível na API

O backend ordena os produtos pelo valor total movimentado e calcula o percentual acumulado. A classe é definida pelo ponto de corte:

Classe	Percentual acumulado	Característica	Controle recomendado
A	até 80%	Poucos itens, grande parte do valor	Contagem frequente, previsão detalhada
B	80% a 95%	Importância intermediária	Controle periódico
C	acima de 95%	Muitos itens, pouco valor	Controle simplificado

● Classificação XYZ

Os produtos são classificados pelo **coeficiente de variação** da demanda ($CV = \text{desvio padrão} \div \text{média}$). Menor CV significa consumo mais previsível. Os cortes usuais na literatura são:

Classe	Coeficiente de variação	Demanda
X	$CV < 0,5$	Estável e previsível
Y	$0,5 \leq CV \leq 1,0$	Variável, com sazonalidade
Z	$CV > 1,0$	Irregular e difícil de prever

● Matriz ABC × XYZ

Na tela de **IA Analítica**, cada produto aparece com sua classe ABC, classe XYZ, giro e valor; a matriz abaixo orienta as ações sugeridas.

	X · estável	Y · variável	Z · irregular
A · maior valor	Estoque de segurança baixo, reposição automática	Monitoramento frequente, margem de segurança	Atenção máxima: alto valor e demanda irregular
B · intermediário	Reposição periódica programada	Revisão periódica	Compra sob demanda com limite
C · menor valor	Lotes grandes, pouco controle	Controle simplificado	Avaliar descontinuação

Figura 5 — Políticas de estoque sugeridas por célula da matriz ABC × XYZ

08 Técnicas do módulo

O portfólio de mineração de dados do StockIQ combina técnicas de previsão, classificação, agrupamento, associação e detecção de padrões incomuns, todas sobre a mesma base de movimentações.

Técnica	Objetivo	Dados utilizados
Previsão de demanda (séries temporais)	Estimar a saída futura por produto/categoria e antecipar rupturas	Movimentações de saída por semana
Curva ABC	Priorizar controle pelos itens de maior impacto financeiro	Valor total movimentado por produto
Classificação XYZ	Medir regularidade da demanda e complementar a ABC	Variabilidade das saídas por produto
Regras de associação (Apriori / Market Basket)	Descobrir produtos movimentados em conjunto para apoiar o slotting	Itens por documento/saída e endereços
Clusterização (k-means)	Agrupar produtos por padrão de consumo e sugerir mínimo/máximo	Média, variância e giro por produto
Detecção de anomalias	Sinalizar ajustes e perdas fora do padrão histórico, cruzando com auditoria	Movimentações de ajuste/perda e <code>audit_logs</code>

Ferramentas

Ferramenta	Papel
Python · pandas · NumPy	Preparação, agregação e modelagem do baseline
scikit-learn	Clusterização e classificação
statsmodels · Prophet	Modelos de séries temporais com sazonalidade
Jupyter Notebooks	Exploração e validação incremental dos modelos
SQLAlchemy · psycopg2	Leitura direta do PostgreSQL operacional
AWS SageMaker	Opção gerenciada para modelos de maior escala

09 Pipeline de dados na nuvem

Para não sobrecarregar o banco transacional com leituras analíticas, a arquitetura-alvo desacopla a operação da análise: cada movimentação publica um evento que alimenta um *data lake* em S3.



Fase inicial: leitura direta do PostgreSQL · evolução: leitura do data lake em S3

Figura 6 — Fluxo de dados do sistema operacional até a inteligência analítica

Benefícios



Desacoplamento

A operação não é afetada pelo volume ou pela complexidade das análises.



Histórico imutável

O data lake preserva os eventos brutos para reprocessar modelos a qualquer momento.

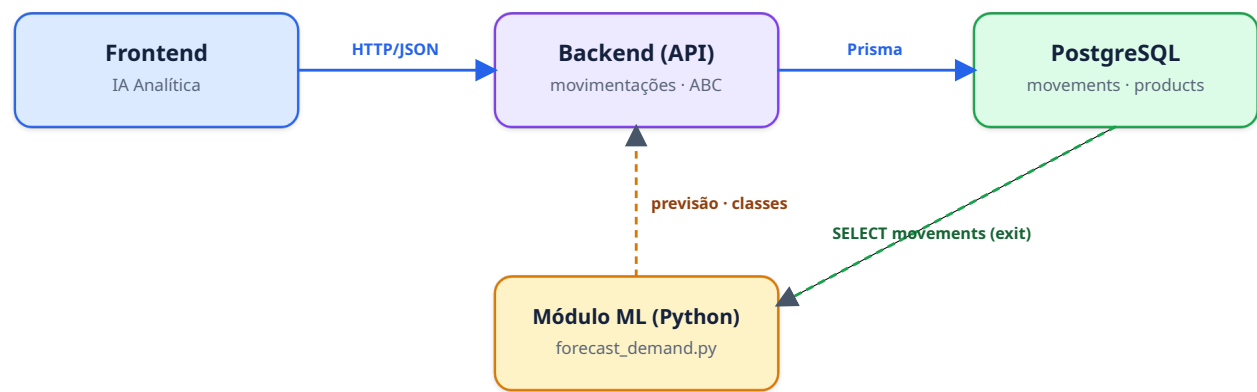


Escalabilidade

Filas e armazenamento gerenciado crescem conforme o volume de movimentações.

10 Integração com o sistema

O módulo de ML participa do ciclo completo do StockIQ: consome o que o backend registra e devolve inteligência para a interface.



ciclo: dados operacionais → modelos → previsões → decisões na interface

Figura 7 — Integração entre ML, backend, banco e interface

Ponto de integração	Descrição
Fonte de dados	Tabela <code>movements</code> (<code>type = 'exit'</code>), unida a <code>products</code> e <code>categories</code> — substitui o CSV em <code>load_sales()</code>
Curva ABC	<code>GET /api/reports/abc</code> e <code>GET /api/dashboard/abc</code> já entregam as classes A, B e C
Reposição	Previsão comparada a <code>Product.minStock</code> e <code>maxStock</code> para sugerir quantidade e urgência
Interface	Tela <code>/dashboard/ia-analitica</code> : gráfico real × previsto, matriz ABC/XYZ, insights e lista priorizada de compras
Alertas	Previsões de alta demanda reforçam os alertas de estoque crítico do módulo de alertas

● Como a previsão vira decisão

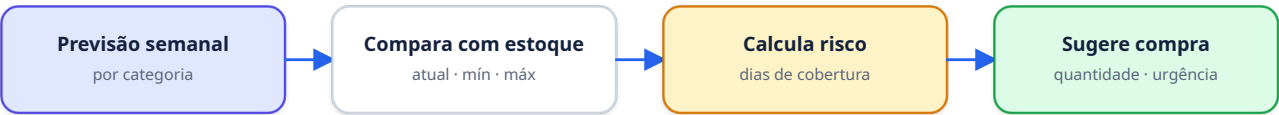


Figura 8 — Da previsão à sugestão de reposição

11 Execução

● Estrutura do módulo

```
machine-learning/
├── data/
│   ├── databases.zip          # dataset original (não versionado)
│   ├── raw/                  # CSVs extraídos (não versionado)
│   └── previsao_demanda.csv   # saída do script (não versionado)
├── src/
│   └── forecast_demand.py     # pipeline de previsão
├── requirements.txt           # pandas ≥ 2.2 · numpy ≥ 1.26
└── README.md
```

● Passo a passo

```
cd machine-learning
python -m venv .env
source .env/bin/activate      # Windows: .env\Scripts\activate
pip install -r requirements.txt

# extrair o dataset (uma vez)
cd data && unzip databases.zip -d raw && cd ..

python src/forecast_demand.py
```

Dependência	Versão	Uso
pandas	≥ 2.2	Leitura, agrupamento e <i>resample</i> temporal
numpy	≥ 1.26	Ajuste linear (<code>polyfit</code>) e vetores

A execução imprime a quantidade de registros carregados, a tabela de previsões por categoria e o caminho do arquivo gerado.

12 Evolução do módulo

O baseline atual estabelece o pipeline completo. O roteiro abaixo descreve como o módulo amadurece para operar sobre os dados reais do StockIQ.



Figura 9 — Roteiro de evolução

Passo	Descrição
1 · Dados do StockIQ	Gerar massa simulada representativa a partir do schema (produtos, categorias, movimentações) e substituir <code>load_sales()</code> por consulta ao PostgreSQL via <code>DATABASE_URL</code> do backend
2 · Comparação de modelos	Contrastar a regressão linear com média móvel e suavização exponencial simples, usando métricas de erro (MAE, RMSE, MAPE) e validação temporal
3 · ABC/XYZ e reposição	Ligar a previsão ao endpoint de curva ABC para sugerir reposição com base em <code>minStock</code> e <code>maxStock</code>
4 · Modelos avançados	Clusterização de produtos, regras de associação, detecção de anomalias e séries com sazonalidade
5 · Nuvem	Pipeline SQS → S3 e execução agendada dos modelos, com resultados servidos à tela de IA Analítica